Multi-Actor E-commerce System

Vendra

Technical Documentation

Ayman Mohamed Mostafa Abdalkawy Omar Wael
Ahmed Yonis Mohamed Nasef

February 2026

Contents

Chapter 1

Introduction

1.1 Purpose of the Document

This document provides comprehensive technical documentation for the **Multi-Actor E-commerce System (Vendra)**. Its purpose is to clearly describe the system architecture, functional and non-functional requirements, technical implementation, and the engineering decisions that shaped the project.

The document serves as:

- A technical reference for developers working on or extending the system
- A structured overview for evaluators and academic reviewers
- A portfolio-quality documentation artefact demonstrating software engineering practices
- A guide for future system extension or backend integration

1.2 Project Overview

The Multi-Actor E-commerce System is a front-end web application that simulates a real-world online marketplace. It is implemented using HTML, CSS, Bootstrap, JavaScript, and Browser Local Storage to demonstrate client-side architecture and role-based system behaviour.

The platform supports three primary actors:

- **Customers** — browse, search, and purchase products
- **Sellers** — manage product listings and process orders
- **Admins** — oversee platform activity and moderate content

Each role operates through a dedicated dashboard with controlled permissions and access restrictions. The system implements:

- Role-based authentication and authorisation
- Product browsing and purchasing workflows
- Inventory and order management

- Administrative supervision and moderation
- Responsive user interface for multiple device sizes

1.3 Scope

This project focuses exclusively on a client-side implementation of an e-commerce system. All data persistence is handled using Browser Local Storage; no backend server or database is integrated.

In Scope

- Role-based authentication and authorisation
- Product catalogue management
- Shopping cart and checkout simulation
- Multi-dashboard architecture (Customer, Seller, Admin)
- Responsive design using Bootstrap 5
- Modular JavaScript structure (Models, Services, UI Components)
- GitHub-based collaborative development
- Deployment as a static site on GitHub Pages

Out of Scope

- Backend server implementation or database integration
- Production-grade cloud hosting infrastructure
- Real payment gateway integration
- Advanced security mechanisms (e.g., JWT tokens, SSL certificates)

1.4 Definitions & Terminology

Actor A system user categorised by role: Customer, Seller, or Admin.

RBAC *Role-Based Access Control* — a security approach in which system access is restricted based on the user's assigned role.

Dashboard
A dedicated interface tailored to a specific role, providing access to role-relevant features only.

Local Storage
A browser-based key-value storage mechanism used to persist application data on the client side.

CRUD Operations
The four fundamental data operations: **C**reate, **R**ead, **U**pdate, and

Delete, applied to entities such as products and users.

State Management

The handling of application data across user interactions using JavaScript and Local Storage.

Seed Data

Pre-loaded example records initialised in Local Storage when the application is first launched, enabling immediate demonstration.

Chapter 2

System Overview

2.1 System Objectives

The Multi-Actor E-commerce System is designed to simulate the core operations of a real-world online marketplace within a fully client-side environment.

The primary objectives are:

- Build a scalable, multi-role e-commerce architecture
- Implement role-based authentication and authorisation
- Simulate marketplace workflows for customers, sellers, and admins
- Deliver a responsive, production-style user interface
- Apply JavaScript (ES6+) for dynamic state and interaction management
- Manage persistent application data using Browser Local Storage

2.2 High-Level Architecture

The system follows a **client-side layered architecture** where all application logic and data management occur within the browser. The architecture is structured into four logical layers:

tableheadbg	
UI Layer	Built using HTML and Bootstrap for layout, components, and responsive design.
tablerowalt Interaction Layer	Implemented with JavaScript to handle user events, form submissions, and navigation.
State Layer	Managed using Browser Local Storage to persist entity data across sessions.
tablerowalt Rendering Layer	Dynamically updates the DOM based on application state without full-page reloads.

Table 2.1: Client-Side Layered Architecture

No backend server or external database is used in this implementation.

2.3 Application Flow

The general application workflow proceeds as follows:

1. User registers or logs into the system via the authentication page.
2. The selected role is validated and stored in Local Storage.
3. Upon successful authentication, the user is redirected to the appropriate role-specific dashboard.
4. Each dashboard validates the stored role on page load before granting access.
5. Users interact with system features according to their role permissions.
6. All updates (users, products, orders, cart items) are persisted to and retrieved from Local Storage.

Access Control: Any unauthorised access attempt — for example, manually navigating to a protected dashboard URL — triggers automatic redirection to the login page.

2.4 Design Principles

The system design is guided by the following core principles:

Modularity
: Separation of concerns between UI, logic, and storage layers; each module encapsulates a single responsibility.

Role Isolation
: Each actor operates within a dedicated dashboard; cross-role access is prevented by front-end guards.

Maintainability
: Structured file organisation with reusable components and well-documented code for ease of future development.

Responsiveness
: Mobile-first layout using Bootstrap grid and utility classes.

Scalability Consideration
: Architecture is structured to allow future backend integration with minimal refactoring.

Chapter 3

Stakeholders & User Roles

3.1 Customers

Customers are end-users who interact with the e-commerce platform primarily to browse and purchase products. Their capabilities include:

- Browse the product catalogue
- View detailed product information
- Add and remove products from the shopping cart
- Complete purchases via the checkout simulation
- View order history
- Manage personal account and profile information

3.2 Sellers

Sellers are registered vendors who manage product listings and handle orders on the platform. Their responsibilities include:

- Add new products to inventory
- Edit or delete existing product listings
- Process and manage incoming customer orders
- Track sales performance and analytics

3.3 Admins

Admins oversee the entire system, ensuring proper operation and content moderation. Their responsibilities include:

- Manage all user accounts (customers and sellers)
- Moderate and approve product listings
- Handle customer support requests
- Monitor overall system activity
- Remove accounts or change user roles as required

3.4 Permissions Matrix

The system enforces Role-Based Access Control (RBAC) to restrict features based on the user's assigned role. Table 3.1 summarises the permission levels.

tableheadbg		
Customer	Limited	Browse catalogue, add to cart, checkout, view order history, manage profile
tablerowalt Seller	Moderate	Manage own products, process orders, view sales analytics
Admin	Full	Manage all users, moderate products, handle support requests, oversee system activity

Table 3.1: Role-Based Permissions Matrix

Chapter 4

Functional Requirements

This chapter describes the detailed functional requirements of the Multi-Actor E-commerce System, organised by user role and core system feature.

4.1 Authentication & Authorisation

- Role-based login and registration (Customer, Seller, Admin)
- Actor role selection during registration
- Automatic dashboard redirection based on authenticated role
- Access restriction for all protected pages and dashboards
- Role validation executed on every page load
- Automatic redirection to the login page on unauthorised access attempts

4.2 Customer Module

- Browse product catalogue displaying images, names, and prices
- Search and filter products by keyword or category
- View detailed product information on dedicated product pages
- Add and remove products from the shopping cart
- Adjust product quantities within the cart
- Complete checkout with shipping and payment forms
- Review order summary and confirm purchase
- View full order history
- Manage personal account and profile information

4.3 Seller Module

- Add new products to inventory with images, descriptions, and pricing
- Edit existing product listings
- Delete products from inventory
- Manage and track incoming customer orders

- View sales performance and analytics dashboards

4.4 Admin Module

- Full access to all platform features
- Manage all registered user accounts (customers and sellers)
- Moderate, approve, or remove product listings
- Remove user accounts or change assigned roles
- Handle customer support requests
- Monitor and review overall system activity

Chapter 5

Non-Functional Requirements

Non-functional requirements define the quality attributes, system constraints, and performance expectations of the Multi-Actor E-commerce System.

5.1 Performance

- Fast loading times for all pages and dashboards
- Efficient rendering of product images using a client-side caching mechanism
- Smooth interaction handling with minimal perceived delays

5.2 Security

- Role-based access control to restrict features per user role
- Input validation of required fields before data persistence
- Client-side password encoding using Base64 (sufficient for a prototype)
- Prevention of unauthorised dashboard access via automatic redirection

5.3 Usability

- Intuitive user interface designed for all roles without a learning curve
- Clear visual feedback via toast notifications for user actions
- Consistent navigation patterns across all dashboards
- Light and Dark theme toggle for personalised user experience

5.4 Responsiveness

- Mobile-first design using the Bootstrap 5 grid system
- Full compatibility across smartphones, tablets, and desktop screens
- Dynamic adjustment of layout and content elements based on viewport size

5.5 Maintainability

- Modular JavaScript architecture separating UI, interaction logic, and state management
- Clear and consistent project structure for easy navigation and future feature additions
- Reusable UI components shared across dashboards and pages
- Extensive code commenting and documentation to support team collaboration

Chapter 6

System Models

Although Vendra is a front-end application, it incorporates data models to simulate backend-like entities. These models encapsulate data structure, validation rules, and core logic, and they interact with service layers to persist data in Local Storage.

6.1 User Model

Represents all system users — Customers, Sellers, and Admins.

- **Required fields:** `name`, `email`, `password`, `role`
- **Optional fields:** phone, address, image, city, state, zipcode, country
- Role validation enforced for values `admin`, `customer`, or `vendor`
- Password encoding using Base64 prior to storage
- Unique identifier (`id`) generated via `crypto.randomUUID()`
- Creation timestamp stored in `createdAt` field

6.2 Product Model

Represents products listed on the platform.

- **Required fields:** `vendorId`, `title`, `price`, `category`, `stock`
- **Optional fields:** description, discount, rating, reviews, weight, images
- Computes `finalPrice` dynamically based on any applicable discount
- Flags products as `featured` based on their rating threshold
- Maintains an `approvalStatus` for admin moderation
- Unique identifier (`id`) and creation timestamp included

6.3 Cart Model

Represents a user's active shopping cart.

- Linked to a specific user via `userId`
- Stores an array of cart item objects (product ID and quantity)

- Tracks cart creation timestamp via `createdAt`

6.4 Order Model

Represents completed or pending customer orders.

- Linked to a user (`userId`) and an associated set of cart items
- Maps product and vendor identifiers for each order line
- Calculates individual item totals and the overall order total
- Tracks order creation date
- Maintains order status (e.g., `pending`, `completed`)
- Validates product existence in the system before creating order entries

6.5 Model Usage Summary

- Models are instantiated to manage entities within the front-end layer
- Services interact with models to add, update, or remove entity records in Local Storage
- Validation ensures required fields and data integrity are enforced before any storage operation
- This design simulates a backend ORM-like pattern within a purely client-side system

Chapter 7

System Services

Services act as an interface layer between models and Local Storage. They simulate backend-like CRUD operations for each entity type, maintaining data integrity and encapsulating all business logic.

7.1 Cart Service

Manages user shopping carts with the following operations:

- Retrieve a cart by user ID
- Add items to an existing cart
- Remove items from a cart
- Clear an entire cart
- Get the total item count in a cart
- Calculate the total price of all cart items
- Persist all cart changes to Local Storage

7.2 Order Service

Manages orders for customers and vendors:

- Create new orders from an active cart
- Retrieve orders by user ID
- Retrieve all orders (Admin-level access)
- Retrieve orders for a specific vendor
- Update order status (e.g., `pending` → `completed`)
- Map cart items to product and vendor IDs; calculate order totals

7.3 Product Service

Manages all product-related operations:

- Create new products (with duplicate title check per vendor)

- Update existing product records
- Delete products from inventory
- Retrieve all products, a product by ID, or products by vendor
- Approve products for listing (Admin action)
- Check product stock availability before purchase

7.4 User Service

Manages user accounts and session data:

- Create new users with unique email validation
- Retrieve users by ID or retrieve all users
- Update user data, cascading changes to the current session if applicable
- Delete user accounts with cascading removal of related carts and products
- Manage the current user session using Local Storage or Session Storage

7.5 Service Layer Summary

- Services interact directly with models to instantiate and manipulate entities
- All persistent data is stored in Local Storage, simulating database-like behaviour
- The service layer provides a structured, maintainable abstraction between the UI and raw data
- Business logic and validation rules are centralised within services, avoiding duplication across UI components

Chapter 8

Data Storage & Model Integration

The Multi-Actor E-commerce System is entirely front-end. Persistent data is stored in **Browser Local Storage**, managed through a centralised `storage` utility module. Models define structure and validation rules, while services interact with the `storage` utility to perform all CRUD operations.

8.1 Storage Utility

The `storage` utility provides generic, reusable methods for managing arrays of records in Local Storage:

`tableheadbg`	
`get(key)`	Retrieve all records for a key (returns an empty array if none exist)
`tablerowalt set(key, value)`	Save an entire array to Local Storage under the given key
`add(key, item)`	Append a single record to an existing array
`tablerowalt update(key, id, newData)`	Update a specific record identified by its `id`
`delete(key, id)`	Remove a record by its `id`
`tablerowalt find(key, id)`	Locate and return a single record by `id`

Table 8.1: Storage Utility Methods

8.2 Integration Flow

- **Models** define entities (`User`, `Product`, `Cart`, `Order`) with validation, defaults, and computed fields.
- **Services** wrap the `storage` utility with entity-specific CRUD operations and business rules.

- Typical integration flows:
  - **User Registration:** Instantiate `User` model → `userService.create()` → `storage.add('users', user)`
  - **Add Product to Cart:** `cartService.addItem(userId, productId, qty)` → `storage.get('carts')` and `storage.set('carts', updatedCarts)`
  - **Place Order:** Instantiate `Order` model → `orderService.create()` → `storage.add('orders', order)`
- Services support cascading operations; for example, deleting a user triggers removal of their associated carts and products.

8.3 Data Structure Reference (JSON)

The following JSON structures represent the data schemas stored in Local Storage for each entity:

```
// Users
[
  { "id": "uuid-...", "name": "Alice", "role": "customer",
    "email": "alice@example.com", "password": "BASE64...",
    "createdAt": "..." },
  { "id": "uuid-...", "name": "Bob",   "role": "vendor",
    "email": "bob@example.com",   "password": "BASE64...",
    "createdAt": "..." }
]

// Products
[
  { "id": "uuid-...", "title": "Laptop", "vendorId": "uuid
   -...",
    "price": 999.99, "category": "Electronics", "stock":
   20,
    "finalPrice": 899.99, "approvalStatus": "approved" }
]

// Carts
[
  { "userId": "uuid-...",
    "items": [ { "productId": "uuid-...", "quantity": 2 }
   ],
    "createdAt": "..." }
]

// Orders
```

```
[
  { "id": "uuid-...", "userId": "uuid-...",
    "items": [ { "productId": "uuid-...", "vendorId": "
   uuid-...",
                "quantity": 2, "itemTotal": 1799.98 } ],
    "totalPrice": 1799.98, "status": "pending", "createdAt
   ": "..." }
]
```

Listing 8.1: Local Storage Data Schemas

8.4 Key Design Points

- The `storage` utility centralises all Local Storage CRUD operations, eliminating direct `localStorage` calls in UI code.
- Models enforce entity validation and structural consistency.
- Services wrap `storage` with entity-specific logic and business rules.
- Cascading and computed operations (e.g., price totals, stock checks) are handled exclusively within services.
- This design simulates a maintainable backend data layer while remaining entirely front-end.

Chapter 9

System Architecture

9.1 Client-Side Layered Architecture

The Multi-Actor E-commerce System follows a layered architecture entirely on the client side. All application logic, state management, and data persistence occur within the browser without any server-side component.

9.1.1 UI Layer

- Responsible for all visual presentation and user interface components
- Uses Bootstrap 5 grid system for responsive, mobile-first layouts
- Supports Light and Dark mode themes for personalised user experience
- Provides role-specific dashboards (Customer, Seller, Admin) with distinct navigation and content

9.1.2 Interaction Layer

- Captures user inputs: clicks, form submissions, and navigation events
- Dispatches actions to the service layer and triggers state updates
- Provides real-time feedback to the user via Bootstrap toast notifications

9.1.3 State Management Layer

- Utilises models (`User`, `Product`, `Cart`, `Order`) to define, validate data structures
- Services interact with models to perform CRUD operations
- Persistent data is stored and retrieved from Local Storage via the centralised `storage` utility

9.1.4 Dynamic Rendering Layer

- Pages and components are rendered dynamically based on current application state
- State changes automatically trigger DOM updates without full-page reloads
- Product image caching is used to reduce repeated network requests and improve performance

Chapter 10

Technology Stack

10.1 Frontend Technologies

HTML5
: Provides semantic markup structure for all pages and components.

CSS3
: Handles custom styling, theme variables (Light/Dark mode), and layout refinements.

JavaScript (ES6+)
: Drives all application logic, interactivity, state management, and DOM manipulation.

10.2 UI Framework & Styling

- **Bootstrap 5** — responsive grid system, pre-built components (cards, tables, forms, modals), and utility classes
- **Custom CSS** — theme switching variables for Light and Dark mode support, and minor design customisations

10.3 State Management

- **Browser Local Storage** — persistent client-side state across browser sessions
- **Service Layer** — acts as a client-side API, abstracting storage operations for each entity
- **Model Layer** — defines entity structure, validation logic, and computed fields

10.4 Design & Prototyping

- **Figma** — used to design and prototype dashboards, product pages, and user flows prior to implementation

10.5 Version Control & Deployment

- **Git** — source code versioning with a feature-branch workflow
- **GitHub** — collaborative development, repository hosting, and pull request management
- **GitHub Pages** — static site hosting for live demonstration deployment

10.6 Project Management

- **Jira** — sprint planning, task tracking, and backlog management using a Scrum framework

Chapter 11

Project Structure

The project is organised to clearly separate role-specific pages, shared components, assets, and system utilities. The directory structure below reflects the actual repository layout.

11.1 Directory Organisation

```
MULTI-ACTOR-E-COMMERCE-SYSTEM/
|
|-- index.html                # Main entry point / landing
   page
|
|-- admin/                    # Admin dashboard pages (HTML
   , JS, CSS)
|-- seller/                   # Seller dashboard pages (
   HTML, JS, CSS)
|-- user/                     # Customer-facing pages (HTML
   , JS, CSS)
|
|-- components/               # UI components
|   |-- user/                 # Customer-specific
   components
|   |-- seller/               # Seller-specific components
|   `-- common/               # Shared components (navbar,
   footer, cards)
|
|-- assets/                   # Static assets (images,
   icons, stylesheets)
|
|-- DataBase/                 # Models, services, and seed
   data
|
|-- utils/                    # Utility scripts (storage.js
   , validators, helpers)
|
```

```
|-- LICENSE                    # MIT License
`-- README.md                  # Project README
   documentation
```

Listing 11.1: Repository Directory Structure

11.2 Component Structure

Shared Components (`components/common/`)
Reused across all dashboards and pages — navigation bars, footers, modals, and product cards.

User Components (`components/user/`)
Customer-specific components for profile management, product browsing, shopping cart, and checkout.

Seller Components (`components/seller/`)
Seller-specific components for product management, order tracking, and the analytics dashboard.

11.3 Data & Model Layer

- `DataBase/` contains model definitions for all system entities (`User`, `Product`, `Cart`, `Order`) and their corresponding service modules
- Services perform CRUD operations via the `storage.js` utility to persist data in Local Storage
- Seed data is included to initialise the system with example products for demonstration purposes

11.4 Utilities

- `storage.js` — centralised Local Storage CRUD utility
- Validator scripts — input validation helpers used by models and services
- General helper functions reused across multiple modules

11.5 Entry Point

`index.html` serves as the application entry point. It presents the landing page where users select their role (Customer, Seller, Admin) before proceeding to login or registration, and subsequently redirects authenticated users to their respective dashboards.

Chapter 12

Data Design

12.1 Data Storage Strategy

- Entirely client-side using **Browser Local Storage**
- Data persists as JSON arrays for each entity type: `users`, `products`, `carts`, `orders`
- Services manage all data access, validation, and cascading updates

12.2 User Data Model

tableheadbg		
`id`	String	UUID generated via `crypto.randomUUID()`
`tablerowalt name`	String	Required
`email`	String	Required; must be unique across all users
`tablerowalt password`	String	Required; Base64-encoded before storage
`role`	Enum	Required; one of `customer`, `vendor`, `admin`
`tablerowalt phone`	String	Optional
`address`	String	Optional
`tablerowalt city`	String	Optional
`state`	String	Optional
`tablerowalt zipcode`	String	Optional
`country`	String	Optional
`tablerowalt image`	String	Optional; URL or Base64 image data
`createdAt`	String	ISO timestamp; auto-generated on creation

Table 12.1: User Entity Data Model

12.3 Role Enumeration

customer
: Can browse the catalogue, make purchases, and manage their personal profile and order history.

vendor
: Can manage product listings, process orders, and view sales analytics.

admin
: Has full platform access: user management, product moderation, support handling, and system monitoring.

12.4 Data Validation Rules

- **Required field enforcement** — models reject entity creation if required fields are absent (e.g., `name`, `email`, `password`, `role` for users)
- **Unique constraints** — enforced at the service level (e.g., email uniqueness, product title uniqueness per vendor)
- **Computed fields** — `finalPrice` and order totals are calculated dynamically before storage
- **Cascading updates** — services handle cascading deletions (e.g., removing a user also removes their carts and products)

Chapter 13

Authentication & Access Control

The Multi-Actor E-commerce System implements role-based authentication and authorisation to ensure users access only the features permitted for their assigned role. All authentication logic is handled on the client side using Local Storage.

13.1 Login Flow

1. User navigates to the login page.
2. User enters their registered email address and password.
3. Credentials are validated against stored user records in Local Storage.
4. If valid, the user's session is stored and they are redirected to the appropriate role-based dashboard.
5. If invalid, a toast notification informs the user of the authentication error.

13.2 Registration Flow

1. User navigates to the registration page.
2. User provides required details: name, email, password, and role selection.
3. The system validates all required fields and enforces email uniqueness.
4. On successful validation, the user is stored in Local Storage via `userService.create()`.
5. A success toast notification confirms the registration.
6. The user is then directed to log in with their registered credentials.

13.3 Role Validation Logic

- Each user has a `role` attribute: `customer`, `vendor`, or `admin`
- Every dashboard page checks the currently stored role before rendering content
- Accessing a dashboard with an incorrect or missing role triggers automatic redirection to the login page
- Role-based logic ensures strict separation of features and UI across all actors

13.4 Route Protection Mechanism

- Protected pages validate the current user session using `userService.getCurrentUser()` on every page load
- Users without a valid session or with an incorrect role are redirected to the login page immediately
- This enforces front-end RBAC (Role-Based Access Control) without requiring a backend server

Chapter 14

UI/UX Design

The Multi-Actor E-commerce System emphasises a clean, responsive, and user-friendly interface. The design was created and prototyped in **Figma**, enabling structured visualisation of dashboards, product pages, and user flows prior to development.

14.1 Responsive Design Strategy

- Mobile-first approach to ensure optimal experience on smartphones
- Flexible grid layout using Bootstrap utility classes, adapting seamlessly to tablet and desktop viewports
- Dynamic adjustment of content and UI elements based on screen width
- Consistent layout patterns for each role's dashboard to reduce cognitive load

14.2 Bootstrap 5 Implementation

- Bootstrap 5 used for rapid responsive layout and component consistency
- Leverages the grid system, utility classes, and built-in components including buttons, cards, tables, forms, modals, and toasts
- Ensures visual consistency across all dashboards and pages
- Minimises custom CSS overhead while retaining flexibility for design adjustments

14.3 Dashboard UI Patterns

- Dedicated dashboards for each role: Customer, Seller, and Admin
- Common navigation components (sidebar, top navigation bar) for consistent feature access
- Modular, card-based layouts for products, orders, and statistics
- Clear, prominent action buttons for CRUD operations with immediate toast-based feedback

14.4 User Experience Considerations

- Toast notifications provide real-time feedback for all user actions (e.g., adding to cart, updating orders, deleting items)
- Light and Dark theme toggle allows personalised visual preferences
- Smooth transitions between pages and UI states enhance perceived interactivity
- Preloaded seed data enables new users to explore the full system without manual setup
- Consistent iconography and typography reinforce readability and visual clarity across all pages

Chapter 15

Features Overview

The Multi-Actor E-commerce System provides a comprehensive feature set for all three actor types. This chapter summarises all implemented features, emphasising role-based access, responsive UI, and enhanced user experience.

15.1 User Authentication

- Role-based login and registration (Customer, Seller, Admin)
- Actor role selection during registration
- Automatic dashboard redirection based on authenticated role
- Access restriction and guard redirects for all protected pages

15.2 Home Page

- Featured products showcase section
- Promotions and platform highlights banner
- Category preview tiles for quick navigation
- Fully responsive layout using Bootstrap 5

15.3 Product Catalogue

- Grid and list view display modes
- Product cards showing image, name, price, and rating
- Add-to-Cart functionality directly from the catalogue
- Keyword search and category filtering

15.4 Product Details Page

- Detailed product description and specifications
- Image gallery with multiple product views
- Pricing and available options display
- Navigation back to the product catalogue

15.5 Shopping Cart

- Add or remove products from the cart
- Quantity adjustment for each cart item
- Auto-calculation of subtotals and grand total
- Order summary preview before checkout

15.6 Checkout Process

- Shipping information collection form
- Payment details input form
- Final order review and confirmation step
- Purchase confirmation notification

15.7 Seller Dashboard

- Add, edit, and delete product listings
- Manage and track incoming customer orders
- Sales performance analytics and insights

15.8 Admin Panel

- Full platform administration access
- User management: view, edit, and remove accounts
- Product moderation: approve or remove listings
- Customer support and complaint handling interface

15.9 Responsive Design

- Mobile-first layout strategy
- Optimised for tablet and desktop viewports
- Bootstrap grid and responsive utility classes throughout

15.10 Enhanced Features

Image Caching

Product images are cached client-side to reduce repeated loading and improve overall performance.

Toast Notifications

Real-time pop-up toasts provide user feedback for actions such as adding to

cart, updating products, or deleting records.

Light / Dark Mode

Two complete UI themes are available; the user's preference is persisted across sessions.

Seeded Data

The system initialises with preloaded example products and accounts, enabling immediate demonstration and testing without manual data entry.

Chapter 16

Engineering Decisions

This chapter explains the key technical decisions made during the development of the Multi-Actor E-commerce System, highlighting the rationale, architectural choices, and inherent trade-offs.

16.1 Local Storage as Data Persistence

Rationale:

- The system is entirely client-side; Local Storage provides persistent data storage without any backend infrastructure
- Simplifies deployment — no server setup is required, enabling a live demo on GitHub Pages
- Stores all users, products, carts, and orders as structured JSON objects

Known Limitations:

- Storage capacity is limited (approximately 5–10 MB per domain)
- Data is browser-specific and cannot be synchronised across devices
- No native relational query capabilities (simulated via service logic)

16.2 Modular JavaScript Architecture

- Strict separation of concerns: Models, Services, and UI Components are distinct modules
- Each module encapsulates logic for a single entity or responsibility
- Improves maintainability, readability, and code reusability across dashboards
- Services provide a clean abstraction layer over Local Storage for all CRUD operations

16.3 Role-Based Dashboard Separation

- Separate, dedicated dashboards for Customers, Sellers, and Admins ensure clear role isolation
- Each dashboard independently validates the user's role on page load
- Modular dashboard architecture enables easy extension or addition of new roles in future iterations

16.4 Design Trade-offs

tableheadbg		
Frontend-Only Architecture	Quick deployment; no server costs	Limited scalability; no backend features
tablerowalt Local Storage	Simple and fast; no configuration needed	No multi-device sync; no server-side validation
Bootstrap 5	Rapid responsive development	Minor customisation requires additional CSS overrides
tablerowalt Seeded Data	Improved demo and test experience	Requires state reset for clean production-like testing
Base64 Password Encoding	Basic obfuscation for prototype use	Not suitable for production; reversible encoding

Table 16.1: Engineering Design Trade-offs

Chapter 17

Testing & Validation

This chapter outlines the testing strategies and validation processes applied to ensure the Multi-Actor E-commerce System functions correctly and enforces role-based access control reliably.

17.1 Manual Testing Strategy

- Conducted end-to-end workflow testing for each user role (Customer, Seller, Admin)
- Verified all CRUD operations for Users, Products, Carts, and Orders
- Checked UI responsiveness across mobile, tablet, and desktop viewports
- Confirmed toast notifications appear correctly for all relevant user actions
- Validated Light/Dark theme switching functions consistently across all pages

17.2 Role-Based Access Testing

- Confirmed that dashboard access is correctly restricted to the appropriate role
- Attempted unauthorised access to protected pages; verified automatic redirection to login
- Verified that user-specific data (orders, carts, products) is visible only to the correct actor
- Ensured admin-only actions (user and product management) are inaccessible to Customers and Sellers

17.3 Functional Testing Checklist

tableheadbg	
Authentication	Login, registration, logout, role selection, invalid credentials
tablerowalt Product Management	Add, edit, delete, view, and approve products
Shopping Cart	Add/remove items, quantity adjustments, total price calculation
tablerowalt Orders	Create orders, view order history, update order status
UI / UX	Responsive layouts, navigation, toast notifications, theme toggle
tablerowalt Data Persistence	Verify Local Storage reflects all CRUD operations correctly

Table 17.1: Functional Testing Checklist

Chapter 18

Installation & Setup

The Multi-Actor E-commerce System is a fully front-end application. No backend installation or package management is required. Follow the steps below to run the project locally.

18.1 Prerequisites

- A modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, or equivalent)
- Visual Studio Code or any preferred code editor (optional)
- **Live Server** extension for VS Code, or any equivalent local HTTP server utility

18.2 Running the Project Locally

1. **Clone the repository:**

```
git clone https://github.com/Eng-Ayman-Mohamed/Multi-Actor-E-commerce-System.git
```

2. **Navigate to the project directory:**

```
cd Multi-Actor-E-commerce-System
```

3. **Open the project in VS Code (optional):**

```
code .
```

4. **Launch using Live Server:**
   - Right-click on `index.html` in the VS Code Explorer
   - Select **Open with Live Server**
   - The application will launch automatically in your default browser

18.3 Live Demo

The system is deployed on **GitHub Pages** and is accessible directly in any modern browser without installation:

Live Demo URL:

https://eng-ayman-mohamed.github.io/Multi-Actor-E-commerce-System/

18.4 Important Notes

- All application data (users, products, carts, orders) is stored in the browser's Local Storage
- Clearing browser data or Local Storage will reset the entire application state, including all seeded data

Chapter 19

Collaboration & Workflow

19.1 Git Workflow Strategy

The team adopted a **feature-branch Git workflow** to keep development organised, traceable, and conflict-free:

- Each feature, bug fix, or enhancement is developed on a dedicated branch
- The `main` branch contains only stable, production-ready code
- Pull requests are created to merge feature branches into `main` after peer review

19.2 Branching Model

Main Branch (`main`)
: Contains production-ready code; serves as the source for the live GitHub Pages deployment.

Feature Branches
: Named descriptively by feature .

Hotfix Branches
: Used for urgent bug fixes and minor patches; merged directly into `main` after testing.

19.3 Code Review Process

- Every pull request is reviewed by at least one additional team member
- Review checks include: code quality, functionality correctness, adherence to the established architecture, and documentation accuracy
- Reviewer feedback is provided as inline comments; all issues must be addressed before the branch is approved for merging

19.4 Team Responsibilities

tableheadbg	
Frontend Development	Implement UI components, dashboards, and all interaction logic
tablerowalt Data & Services	Implement models, service layers, and Local Storage integration
UI/UX Design	Design responsive layouts and user flows; Figma used for prototyping
tablerowalt Testing & QA	Execute functional and role-based tests; verify data persistence
Documentation	Maintain comprehensive project documentation for development and evaluation

Table 19.1: Team Responsibilities

Chapter 20

Project Timeline

The project followed a **sprint-based Scrum timeline** managed in Jira. Two sprints were completed, each with defined goals, tasks, and deliverables.

Jira Board:
https://engaymanmohamedabotahakasim-1770972863554.atlassian.net/jira/software/projects/MAECSS/boards/34

20.1 Sprint 1 — Week 1: Foundations

Sprint Goal: Complete foundational authentication and UI elements for all roles (Customer, Seller, Admin), including Light/Dark mode toggle and responsive layout. Focus on user authentication, responsive design, and core Customer-facing pages.

Tasks

Authentication
Implement login, registration, and password reset for all roles. Set up role-based permissions. Create responsive authentication UI. Integrate Light/Dark mode toggle.

Global Layout & Responsive Design
Implement mobile-first layout across the platform. Ensure responsive design for all pages with Light/Dark mode support.

Customer Pages
Home page with search bar and featured products. Product catalogue with basic category filters. Product details pages with images and descriptions.

Testing & QA
Validate authentication flow, responsiveness across viewports, and Light/Dark mode toggle functionality.

Deliverables

- Fully implemented user authentication system

- Responsive layout with working Light/Dark mode toggle
- Customer-facing pages: Home, Product Catalogue, Product Details
- Initial QA pass on authentication and responsiveness

20.2 Sprint 2 — Week 2: Full Feature Completion

Sprint Goal: Complete Seller and Admin functionalities, finalise all Customer features (cart and checkout), and ensure platform-wide responsiveness and Light/Dark mode consistency.

Tasks

Seller Dashboard
: Product management UI (add, edit, manage inventory). Order management and tracking UI. Basic sales analytics (revenue and order counts).

Admin Panel
: User management for all roles. Content moderation (products and reviews). Basic reporting of user and sales data.

Customer Cart & Checkout
: Shopping cart with add/remove and quantity adjustment. Total price calculation and checkout process (address, payment, order summary). Responsive design and Light/Dark mode applied.

Testing & QA
: End-to-end role-based testing. Responsiveness and theme toggle verification. Final QA for Seller Dashboard, Admin Panel, and Customer checkout.

Deliverables

- Seller Dashboard with product management, order tracking, and analytics
- Admin Panel with user management, content moderation, and reporting
- Fully implemented Customer cart and checkout functionality
- Fully responsive platform with consistent Light/Dark mode across all pages
- Final QA completed and sign-off achieved

20.3 Jira Scrum Setup Summary

tableheadbg	
Epics	Customer, Seller, Admin
tablerowalt Sprint 1 Tasks	Authentication, Global Layout, Customer Home & Product Pages
Sprint 2 Tasks	Seller Dashboard, Admin Panel, Customer Checkout, Role-Based Testing

Table 20.1: Jira Scrum Configuration

Chapter 21

Limitations

While the Multi-Actor E-commerce System (Vendra) is fully functional as a front-end prototype, several limitations are inherent to the chosen design and architecture.

21.1 Security Constraints

- Client-side authentication is not secure for production use; session data stored in Local Storage can be manipulated via browser developer tools
- Passwords are encoded with Base64, which is reversible and not suitable for protecting sensitive user credentials
- RBAC relies entirely on front-end validation and Local Storage values, which can be altered by technically proficient users

21.2 Local Storage Constraints

- All persistent data is stored in the browser's Local Storage, constrained by typical browser quotas of 5–10 MB per domain
- Clearing browser data or Local Storage will erase the entire application state, including user accounts and product listings
- Data is browser-specific; multi-device synchronisation is not possible without a backend server

21.3 Frontend-Only Architecture Constraints

- No backend server or database; all business logic and persistence are client-side only
- Real-time updates and multi-user concurrency are not supported
- No integration with external APIs or live payment gateways
- Scalability is limited; performance may degrade with very large datasets in Local Storage

21.4 Summary

These limitations are acceptable within the scope of a demonstration and learning-focused prototype. They clearly define the boundaries of the current system and identify the primary areas targeted by future enhancement efforts, particularly backend integration, secure authentication, and cloud-based data management.

Chapter 22

Future Enhancements

Vendra is architected as an extensible front-end prototype. The following enhancements define a roadmap for evolving the system into a production-ready e-commerce platform.

22.1 Backend Integration

- Introduce a server-side backend to handle data persistence, authentication, and authorisation securely
- Enable multi-user data synchronisation and real-time updates
- Implement RESTful APIs for all CRUD operations and dashboard management

22.2 Database Implementation

- Replace Local Storage with a relational (e.g., PostgreSQL) or NoSQL (e.g., MongoDB) database for scalable data management
- Support advanced queries for product search, order history, and analytics
- Ensure data integrity, automated backups, and recovery mechanisms

22.3 Payment Gateway Integration

- Integrate secure, production-grade payment gateways (e.g., Stripe, PayPal) for real financial transactions
- Implement full checkout workflows with real-time payment confirmation
- Support multiple payment methods and multi-currency options

22.4 Advanced Authentication

- Implement secure password hashing using bcrypt or Argon2
- Introduce OAuth 2.0 / JWT-based authentication for robust, stateless session management
- Enable multi-factor authentication (MFA) for sensitive operations

22.5 Feature Expansion Roadmap

- Add verified product reviews and ratings with admin moderation tools
- Implement customer wishlist and saved-items functionality
- Provide vendor analytics dashboards with interactive sales trend charts and exportable reports
- Introduce in-app notifications, email alerts, and a live customer support chat module
- Enhance UI/UX with animated transitions and advanced product filtering and sorting options

22.6 Summary

These planned enhancements provide a clear and prioritised roadmap to evolve Vendra from a front-end prototype into a scalable, secure, and fully functional e-commerce platform suitable for real-world commercial deployment.

Chapter 23

License

The Multi-Actor E-commerce System (Vendra) is released under the **MIT License**.

MIT License

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so.

The software is provided *"as is"*, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall the authors or copyright holders be liable for any claim, damages, or other liability arising from the use of or other dealings in the Software.

For the full official license text, see:
https://opensource.org/licenses/MIT

Chapter 24

Conclusion

The Multi-Actor E-commerce System (Vendra) demonstrates a fully front-end, role-based marketplace application supporting Customers, Sellers, and Admins within a clean, modular, and maintainable architecture.

Key Achievements

- A modular and maintainable JavaScript architecture (Models, Services, UI Components)
- Role-based authentication and complete dashboard separation per actor
- Dynamic product catalogue with full shopping cart and checkout simulation
- Responsive UI supporting both Light and Dark themes across all viewports
- Full CRUD functionality for all entities, simulated entirely via Local Storage
- Enhanced user experience through toast notifications and image caching
- Deployment-ready static site hosted on GitHub Pages for live demonstration
- Structured collaborative development using Git, GitHub, and Jira Scrum

Final Remarks

Vendra serves as a portfolio-quality demonstration of front-end engineering, modular application design, and role-based system architecture. The project is deliberately structured to allow future integration with backend services, relational or NoSQL databases, and production-grade features such as live payment processing and advanced authentication mechanisms — providing a solid foundation for continued development and real-world deployment.